

Data Structures and Algorithms

Data structures organize data for efficient access.

Time Complexity: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

Linear Structures: Arrays, Linked Lists, Stacks, Queues

Arrays: $O(1)$ access, $O(n)$ insertion/deletion

Linked Lists: $O(n)$ access, $O(1)$ insertion/deletion

Stacks: LIFO (Last In First Out)

Queues: FIFO (First In First Out)

Trees: Binary Search Tree, AVL Tree, Red-Black Tree

BST: $\text{Left} < \text{Root} < \text{Right}$, $O(\log n)$ average operations

Heaps: Complete binary tree with heap property

Graphs: Vertices and Edges, Adjacency Matrix/List

Sorting Algorithms:

- Merge Sort: $O(n \log n)$, stable
- Quick Sort: $O(n \log n)$ average, fastest in practice
- Heap Sort: $O(n \log n)$, in-place

Search Algorithms:

- Binary Search: $O(\log n)$, requires sorted data
- Hash Table: $O(1)$ average case

Dynamic Programming: Overlapping subproblems + Optimal substructure

References: Cormen et al., Introduction to Algorithms